

Valores, expressões e tipos

- Os **valores** são as entidades básicas da linguagem Haskell. São os elementos atômicos.
- Uma **expressão** ou é um valor ou resulta de aplicar funções a expressões.
- O **interpretador** atua como uma calculadora: *lê uma expressão, calcula o seu valor e apresenta o resultado.*

```
> 5.3 + 7.2 * 0.1
6.02
> 2 < length [4,2,5,1]
True
> not True
False
```

- Um **tipo** é um nome que denota uma coleção de valores.
- Se da avaliação de uma expressão **e** resultar um valor do tipo **T**, então dizemos que a **expressão e tem tipo T**, e escrevemos

e :: T

Por exemplo,

```
> :type not True
not True :: Bool
```

Tipos

- Um tipo é um nome que denota uma coleção de valores. Por exemplo,
 - O tipo **Bool** contém os dois valores lógicos: **True** e **False**
 - O tipo **Integer** contém todos os números inteiros: ..., -2, -1, 0, 1, 2, 3, ...
- As funções só podem ser aplicadas a argumentos de tipo adequado. Por exemplo,

```
> 2 + True  
error: ...
```

Porque + deve ser aplicada a números e True não é um número.

- Se não houver concordância entre o tipo das funções e os seus argumentos, o programa é **rejeitado pelo compilador**.

Tipos

- Toda a expressão Haskell bem formada tem um tipo que é automaticamente calculado em tempo de compilação por um mecanismo chamado *inferência de tipos*.
- Por isso se diz que a linguagem Haskell é *“statically typed”*.
- Todos os *erros de tipo* são encontrados em tempo de compilação, o que torna os programas mais robustos.
- Os tipos permitem assim programar de forma mais produtiva, com *menos erros*.
- Num programa Haskell *não é obrigatório escrever os tipos*, o compilador infere-os, mas é boa prática escrevê-los pois é uma forma de documentar o código.

Tipos básicos

Bool	Booleanos	<code>True, False</code>
Char	Caracteres	<code>'a', 'b', 'A', '3', '\n', ...</code>
Int	Inteiros de tamanho limitado	<code>5, 7, 154243, -3452, ...</code>
Integer	Inteiros de tamanho ilimitado	<code>2645611806867243336830340043, ...</code>
Float	Números de vírgula flutuante	<code>55.3, 23E5, 743.2e12, ...</code>
Double	Números de vírgula flutuante de dupla precisão	<code>55.3, 23.5E5, ...</code>
()	<i>Unit</i>	<code>()</code>

Tipos compostos

- **Tuplos** - sequências de tamanho fixo de elementos de diferentes tipos

```
(5, True) :: (Int, Bool)
```

```
(8, 3.5, 'b') :: (Int, Float, Char)
```

- **Listas** - sequências de tamanho variável de elementos de um mesmo tipo

```
[1,2,3,4,5] :: [Int]
```

```
[True, True, False] :: [Bool]
```

- **Funções** - mapeamento de valores de um tipo (o domínio da função) em valores de outro tipo (o contra-domínio da função)

```
fact :: Integer -> Integer
```

```
fact 0 = 1
```

```
fact n = n * fact (n-1)
```

Tuplos

Um tuplo é uma sequência de tamanho fixo de elementos que podem ser de diferentes tipos.

$(T_1, T_2, \dots, T_n)$ é o tipo dos tuplos de tamanho n , cujo 1º elemento é de tipo T_1 , o 2º elemento é de tipo T_2 , ..., e na posição n tem um elemento de tipo T_n .

Exemplos:

```
('C', 2, 'A') :: (Char, Int, Char)
```

```
(True, 1, False, 0) :: (Bool, Int, Bool, Int)
```

```
(3.5, ('a', True), 20) :: (Float, (Char, Bool), Int)
```

Listas

As listas são sequências de tamanho variável de elementos do mesmo tipo.

[T] é o tipo das listas de elementos do tipo T.

Exemplos:

```
[10,20,30] :: [Int]
[10, 20, 6, 19, 27, 30] :: [Int]
[True,True,False,True] :: [Bool]
[( 'a' ,True), ( 'b' ,False)] :: [(Char,Bool)]
[[3,2,1], [4,7,9,2], [5]] :: [[Int]]
```

Funções

Uma função é um mapeamento de valores de um tipo (o [domínio](#) da função) em valores de outro tipo (o [contra-domínio](#) da função)

$T_1 \rightarrow T_2$ é o tipo das funções que *recebem* valores do tipo T_1 e *devolvem* valores do tipo T_2 .

Exemplos:

```
even :: Int -> Bool
odd  :: Int -> Bool
not  :: Bool -> Bool
```


Funções com vários argumentos

Uma função com vários argumentos pode ser codificada de duas formas.

- Usando tuplos:

`soma` recebe um par de inteiros `(x,y)` e devolve o resultado inteiro `x+y`.

```
soma :: (Int,Int) -> Int
soma (x,y) = x + y
```

- Retornando funções como resultado:

`add` recebe um inteiro `x` e devolve uma função `(add x)`.
Depois esta função recebe o inteiro `y` e devolve o resultado `x+y`.

```
add :: Int -> (Int -> Int)
add x y = x + y
```

Curried functions

```
soma :: (Int, Int) -> Int  
add  :: Int -> (Int -> Int)
```

soma e add produzem o mesmo resultado final, mas soma recebe os dois argumentos ao mesmo tempo, enquanto add recebe um argumento de cada vez.

- As funções que recebem os seus argumentos um de cada vez dizem-se “*curried*” em honra do matemático Haskell Curry que as estudou.
- Funções que recebem mais do que dois argumentos podem ser *curried* retornando funções aninhadas.

```
mult :: Int -> (Int -> (Int -> Int))  
mult x y z = x * y * z
```

`mult` recebe um inteiro `x` e devolve uma função `(mult x)`, que por sua vez recebe o inteiro `y` e devolve a função `(mult x y)`, que finalmente recebe o inteiro `z` e devolve o resultado `x*y*z`.

Porque é que as funções *curried* são úteis?

- As funções *curried* são mais flexíveis porque é possível gerar novas funções, **aplicando parcialmente** uma função *curried*.

```
add 1 :: Int -> Int
```

- As funções Haskell são normalmente definidas na forma *curried*.

```
take :: Int -> [Int] -> [Int]  
take 5 :: [Int] -> [Int]
```

Convenções

Para evitar o uso excessivo de parêntesis quando se usam funções *curried* são adotadas as seguintes convenções:

- A **seta ->** associa à **direita**

```
Int -> Int -> Int -> Int
```

Significa `Int -> (Int -> (Int -> Int))`

- A **aplicação** de funções associa à **esquerda**

```
mult x y z
```

Significa `((mult x) y) z`

Funções polimórficas

Há funções às quais é possível associar mais do que um tipo concreto. Por exemplo, a função `length` (que calcula o comprimento de uma lista) pode ser aplicada a quaisquer listas independentemente do tipo dos seus elementos.

```
> length [3,2,2,1,4]
5
> length [True,False]
2
```

Um função diz-se **polimórfica** se o seu tipo contém **variáveis de tipo** (representadas por letras minúsculas).

```
length :: [a] -> Int
```

Para qualquer tipo `a`, a função `length` recebe uma lista de valores do tipo `a` e devolve um inteiro.

Funções polimórficas

As variáveis de tipo podem ser instanciadas por diferentes tipos consoante as circunstâncias.

```
length :: [a] -> Int
```

```
> length [3,2,2,1,4]  
5
```

a = Int

```
> length [True,False]  
2
```

a = Bool

- As variáveis de tipo começam por letras minúsculas (normalmente usam-se as letras a, b, c, etc).
- O nome dos tipos concretos começa sempre por uma letra maiúscula (ex: Int, Bool, Float, etc)

Funções polimórficas AULA 2

A maioria das funções da biblioteca `Prelude` são polimórficas.

```
head :: [a] -> a
tail :: [a] -> [a]
take :: Int -> [a] -> [a]
fst  :: (a,b) -> a
snd  :: (a,b) -> b
id   :: a -> a
reverse :: [a] -> [a]
zip  :: [a] -> [b] -> [(a,b)]
```

Sobrecarga (*overloading*) de funções

Considere os seguintes exemplos:

```
> 3 + 2
5
> 10.5 + 1.7
12.2
```

Qual será o tipo da soma (+) ?

Note que `a -> a -> a` é um tipo demasiado permissivo para a função (+), pois não é possível somar elementos de qualquer tipo

```
> 'a' + 'b'
:error ...
```

O Haskell resolve o problema com **restrições de classes**

```
(+) :: Num a => a -> a -> a
```

Para qualquer tipo numérico `a`, a função (+) recebe dois valores do tipo `a` e devolve um valor do tipo `a`.

Sobrecarga (*overloading*) de funções

Uma função polimórfica diz-se **sobrecarregada** se o seu tipo contém uma ou mais restrições de classes.

```
(+) :: Num a => a -> a -> a
```

```
> 3 + 2
5
> 10.5 + 1.7
12.2
> 'a' + 'b'
error: ...
```

a = Int

a = Float

Char não é um
tipo numérico

Mais exemplos:

```
sum :: Num a => [a] -> a

(*) :: Num a => a -> a -> a

product :: Num a => [a] -> a
```

Class constraints

O Haskell tem muitas classes mas, por agora, apenas precisamos de ter a noção de que existem as seguintes

Num - a classe dos tipos numéricos (tipos sobre os quais estejam definidas operações como a soma e a multiplicação).

Eq - a classe dos tipos que têm o teste de igualdade definido.

Ord - a classe dos tipos que têm uma relação de ordem definida sobre os seus elementos.

```
(+) :: Num a => a -> a -> a
```

```
(==) :: Eq a => a -> a -> Bool
```

```
(<) :: Ord a => a -> a -> Bool
```

Mais tarde estudaremos em mais detalhe o mecanismo de classes do Haskell.

!!! Aviso !!!

Na versão actual do GHCi, se perguntarmos o tipo de certas funções sobre listas temos uma surpresa!!

```
> :type sum  
sum :: (Foldable t, Num a) => t a -> a
```

Este é um tipo mais geral do que `sum :: Num a => [a] -> a` mas como as listas pertencem à classe `Foldable`, quando aplicamos `sum` a uma lista de números, este é o tipo efetivo da função `sum`.

Ao longo das aulas iremos sempre apresentar o tipo destas funções usando **listas** em vez da classe `Foldable`, para simplificar a apresentação.

Sempre que virem a classe `Foldable` num tipo das funções do Prelude, podem entender isso como sendo o tipo das listas.